

RESIDENT EVIL AI AGENT

Full Technical Specification & Implementation Guide

RL Systems | Computer Vision | LLM Integration | Live Dashboard

v1.0 — 2025

1. Executive Summary

This document is the complete technical specification for an AI Agent system that plays Resident Evil games autonomously — learning through reinforcement learning, reasoning through Large Language Models, and improving continuously across every session. The system is designed to run live on a Windows PC alongside the game, with a real-time GUI dashboard that makes every layer of the agent's thinking visible and expandable.

The architecture integrates three complementary AI technologies into a single unified pipeline:

- Reinforcement Learning (RL) — the agent learns reflexive, low-level behaviors (combat movement, dodging, aiming) through millions of trial-and-error game interactions, continuously improving its action policy
- Computer Vision — a real-time perception pipeline processes every game frame to extract structured information: enemy positions, health/ammo values, item locations, door states, and HUD elements
- Large Language Models (LLMs) — a strategic brain that receives annotated screenshots and game state summaries at key decision moments, then issues high-level goals and recommendations that guide the RL agent

Target games: RE2 Remake, RE3 Remake, RE4 Remake, and RE Village. All run on PC via Steam and support Borderless Windowed mode — a prerequisite for screen capture.

Core Design Principle: The agent does not hard-code any game knowledge. It learns everything purely from visual observation — exactly as a human player would — but iterates on experience continuously, running 24 hours a day if needed, accumulating strategic knowledge that persists across sessions.

2. System Architecture Overview

2.1 The Three-Layer Hierarchy

The agent operates as a three-layer hierarchy. Each layer handles a different time scale of decision-making — mirroring how expert human RE players actually approach the games: act fast in the moment, think about the room, plan across the dungeon.

Layer	Component	Decision Frequency	Responsibility
Strategic	LLM Brain (GPT-4o / Claude 3.7)	Every 3–10 sec or on trigger events	Route planning, puzzle solving, inventory management, long-term objectives
Tactical	RL Goal Manager + Event Detector	Every 0.5–1 second	Sub-goal pursuit, room navigation, threat classification, action sequencing
Operational	RL Action Agent (PPO)	Every frame (16–33 ms)	Movement, aiming, shooting, dodging, item pickup, interact

2.2 Data Flow — Every Game Tick

1. mss captures the game screen at up to 60 fps from a borderless windowed session
2. Perception Pipeline (YOLOv11 + EasyOCR + OpenCV) annotates the frame: bounding boxes on enemies/items/doors, health/ammo values from HUD, room ID from template matching
3. Annotated frame + structured state vector are assembled into the RL observation
4. RL Action Agent (PPO) selects a MultiDiscrete action: move direction, camera, combat, interact, evasion
5. pydirectinput executes the action as real DirectX-level keyboard and mouse input to the game
6. Simultaneously, the Event Detector monitors for trigger conditions and fires async LLM calls when thresholds are crossed
7. LLM Brain receives screenshot + structured context, reasons through the situation, returns high-level goal + inventory recommendation as structured JSON
8. New goal is encoded as a vector and injected into the RL agent's observation, steering its behavior toward the LLM's recommended direction
9. All events, rewards, screenshots, and LLM outputs are logged asynchronously to SQLite and streamed to the live GUI dashboard

Key Architectural Constraint: The LLM call runs in a separate thread and NEVER blocks the RL loop. The RL agent always operates at full speed using the most recently available goal from the LLM. A stale goal is always better than a frozen agent waiting for an API response.

3. Technology Stack

3.1 Screen Capture & Game Control

Screen Capture: mss

mss (Multiple Screen Shots) is the recommended capture library. It accesses the Windows GDI directly, achieving 60 fps+ with minimal CPU overhead, and outputs NumPy arrays natively — eliminating the conversion step that slows down other capture tools like OpenCV's VideoCapture.

- Library: mss >= 9.0
- CRITICAL setup: The game MUST run in Borderless Windowed mode. mss cannot reliably capture DirectX Exclusive Fullscreen. Set this in-game under Options > Display > Window Mode > Borderless.
- Capture the specific game window by its pixel coordinates (not the full desktop) so the agent works identically at 1080p and 1440p
- Output format: BGRA NumPy array. Drop the alpha channel (frame[:, :, :3]) to get BGR immediately after capture

```
# Target window capture — always use specific coordinates, not full screen
import mss, numpy as np
GAME_REGION = {'top': 0, 'left': 0, 'width': 1920, 'height': 1080} # from
config.yaml
with mss.mss() as sct:
    frame_bgra = np.array(sct.grab(GAME_REGION))
    frame_bgr = frame_bgra[:, :, :3] # drop alpha channel
```

Input Control: pydirectinput

CRITICAL — Do NOT use pyautogui for DirectX games.

pyautogui uses Win32 PostMessage/SendMessage which DirectX games ignore by design. pydirectinput uses SendInput() via ctypes — the only input injection method that works at the DirectX level. This is the most common setup failure for game AI projects on Windows.

win32api can read window state (for coordinate math) but must NOT be used for sending game inputs.

- Library: pydirectinput >= 1.0.4
- Handles WASD movement, all button presses, and relative mouse movement for camera control
- For analog-style camera look: call pydirectinput.moveRel(dx, dy) in small increments (± 10 – 20 pixels) each frame rather than large jumps
- Add a short pause (0.02s) after each pydirectinput call — DirectX game input queues can drop events if flooded

3.2 Game State Perception Pipeline

Object Detection: YOLOv11

YOLOv11 (Ultralytics, 2025) is the current best-in-class real-time object detector. Use the nano (n) or small (s) model for real-time GPU inference at gaming frame rates.

- Library: ultralytics ≥ 8.3 (YOLOv11 ships within this package — model name: 'yolo11n.pt' or 'yolo11s.pt')
- Run inference on GPU (CUDA). YOLOv11n achieves ~3–5 ms per frame on an RTX 3080 — well within the 16 ms budget for 60 fps
- Initial detection: Use the COCO-pretrained model. COCO's 'person' class detects humanoid enemies; architectural classes detect doors; everyday objects detect items with some noise
- Progressive improvement: As the agent plays, collect RE-specific frames and annotate them (Labelling tool) to fine-tune a custom RE model. Start fine-tuning after ~300 labeled images.
- Classes to detect: enemies (zombie, ganado, molded, lycan), loot pickups, interactive objects (doors, levers, safes, combination locks), boss characters, player character

HUD Reading — OCR: EasyOCR

Correction from original draft: Tesseract was listed as the primary OCR option. Tesseract is designed for scanned document fonts on white backgrounds — it performs poorly on stylized game UI fonts with colored or semi-transparent HUD overlays. EasyOCR uses a deep learning backbone and handles RE's HUD fonts significantly better out of the box with no custom training required.

- Library: easyocr ≥ 1.7
- Reads: health percentage, ammo count, item names in inventory popups, map room labels
- CRITICAL optimization: Crop the specific HUD regions (bottom-left for health, bottom-right for ammo) BEFORE passing to EasyOCR. Running OCR on the full 1920x1080 frame would be 20x slower than on a cropped 200x50 HUD patch.
- Cache OCR results: compute a pixel hash of the HUD region each frame; only re-run EasyOCR when the hash changes. Health bars update at most a few times per second.

UI State Detection: OpenCV Template Matching

- Library: opencv-python ≥ 4.10
- Template matching (cv2.matchTemplate) identifies: current room by matching known room background samples, UI state (inventory open, map open, cutscene active, death screen, loading screen), puzzle interaction UI elements
- CRITICAL: The agent MUST pause training during loading screens, cutscenes, and death animations. Build a non-interactive state detector that returns a zeroed observation and zero reward for these states without advancing the step counter.
- Template database: a JSON-indexed directory of PNG templates per game, loaded at startup from config

- Matching threshold: use `cv2.TM_CCOEFF_NORMED` with a threshold of 0.85 — below this, treat as unmatched to avoid false positives

3.3 The RL Agent

Primary Framework: Stable-Baselines3 + PPO

Stable-Baselines3 (SB3) is the industry standard for reliable, well-tested RL implementations. Proximal Policy Optimization (PPO) is the optimal algorithm choice here: it is sample-efficient on image observations, handles MultiDiscrete action spaces natively, is numerically stable over long training runs, and has extensive documentation and community support.

- Libraries: `stable-baselines3 >= 2.3`, `sb3-contrib >= 2.3`
- Policy: `RecurrentPPO` from `sb3-contrib` — gives the agent an LSTM cell so it naturally handles sequential game context (what happened 5 steps ago matters in RE) without requiring manual frame stacking in the observation
- Frame stack: still include 4 stacked frames in the observation for raw motion cues, even with `RecurrentPPO`
- Hardware requirement: CUDA-capable GPU. Minimum RTX 3070 (8 GB VRAM). RTX 3080/4070 (12 GB VRAM) recommended for comfortable headroom.

Advanced Option: DreamerV3 (World-Model RL)

DreamerV3 builds an internal model of the game world and plans ahead through imagination — highly effective for complex environments with sparse rewards like RE. It is the state-of-the-art for visual game RL.

- Repository: `danijar/dreamerv3` on GitHub
- Requires ~16 GB VRAM and a more complex training setup. Strongly recommended to validate the full pipeline with SB3/PPO first before migrating to DreamerV3.

3.4 LLM Integration — The Strategic Brain

Model Assignment Strategy

With access to GPT-4o, Claude 3.7 Sonnet, and Grok-2, the optimal strategy assigns each model to the task that matches its specific strengths:

Model	Provider	Role in Agent	Why This Model
GPT-4o	OpenAI	Primary strategic brain — real-time vision + reasoning	Best multimodal vision at speed; understands spatial layouts in screenshots
Claude 3.7 Sonnet	Anthropic	Complex puzzle solving and long-horizon planning	Strongest at structured reasoning chains; best JSON output compliance

Model	Provider	Role in Agent	Why This Model
Grok-2	xAI	Fast secondary validation + narrative commentary	Lower latency; good for quick anomaly checks and dashboard commentary

LLM Orchestration: LangGraph

LangGraph (by LangChain) provides a graph-based workflow engine for multi-step LLM pipelines. It manages the decision workflow between trigger detection, model selection, prompt building, API calling, response parsing, and goal injection.

- Library: langgraph >= 0.2
- Workflow graph: Observe State -> Classify Trigger Type -> Select LLM Model -> Build Context Prompt -> Call LLM API (async) -> Parse JSON Response -> Validate Against Game State -> Inject Goal into RL Observation -> Log to SQLite
- All LLM calls use structured output mode (response_format: {type: 'json_object'}) to guarantee parseable responses
- Fallback: if JSON parse fails or confidence < 0.4, the previous goal persists — the agent never freezes waiting for a valid LLM response

LLM Call Triggers — Critical Design Rule

CRITICAL: LLMs must NOT be called on every frame.

A GPT-4o vision call takes 2–8 seconds. At 60 fps, calling every frame would cost approximately \$3,600/hour and completely freeze the agent. The correct pattern is event-triggered calls with a minimum 3-second cooldown between any two calls.

LLM calls fire on these specific events:

- New room detected (template match changed to an unseen room hash): Always triggers an LLM call
- Puzzle UI detected: Always triggers Claude 3.7 specifically for puzzle-solving reasoning
- Health drops below 25%: Triggers LLM to assess threat level and recommend healing or escape strategy
- Agent stuck: No new rooms, items, or events for 30 consecutive seconds
- Inventory full: LLM decides which items to drop, combine, or store based on current objectives
- Boss phase transition: HP bar template match detects a phase change
- Manual override: User can force an LLM call at any time via dashboard button

Prompt Template Design

Every LLM call sends three components: a constant system prompt, a dynamic context JSON block, and the annotated current screenshot.

SYSTEM PROMPT (constant across all calls):
You are the strategic brain of an autonomous AI agent playing Resident Evil. Analyze the screenshot and game state, then provide concise, actionable guidance. You must respond ONLY with valid JSON matching the required schema.

CONTEXT BLOCK (dynamic JSON injected per call):
{ "current_room": str, "health_pct": 0-100, "ammo": { "primary": int, "secondary": int }, "inventory": [str], "recent_events": [last 10 events], "known_objectives": [str], "steps_since_progress": int, "detected_enemies": [{ "type": str, "distance": float }] }

REQUIRED JSON RESPONSE SCHEMA:
{ "threat_level": 0-10, "immediate_goal": str, "rl_sub_goal": str, "inventory_action": str|null, "reasoning": str, "confidence": 0.0-1.0 }

3.5 Memory & Knowledge System

Two-Tier Memory Architecture

The memory system has two tiers: a short-term in-memory ring buffer for the RL agent's recent context, and a long-term persistent SQLite knowledge base for cross-session learning.

Correction from original draft: JSON was listed as an alternative storage option alongside SQLite. JSON files require full parse-and-rewrite on every update, degrade to unusable speeds past ~500 records, and cannot support queries. SQLite is the only correct choice — it is lightweight, zero-configuration, supports concurrent async reads/writes, and can handle millions of event rows with sub-millisecond query times.

SQLite Schema

Table	Key Columns	Purpose
rooms	id, name, game, first_visited, visit_count, connected_rooms JSON, template_hash	Graph of all discovered rooms; powers the map panel in dashboard
items	id, name, room_id, position_x, position_y, collected, timestamp	Every item seen or collected; fed to LLM context for inventory planning
enemies	id, type, room_id, killed, times_encountered, screenshot_path, timestamp	Enemy encounter history; informs LLM threat assessment
events	id, type, description, room_id, reward_delta, timestamp, screenshot_path	Complete chronological event log; used for dashboard Event Log panel
puzzles	id, room_id, description, solved, solution_used, llm_reasoning, timestamp	Puzzle encounter log with LLM solution reasoning preserved
agent_state	timestamp, health, ammo_primary, current_room, current_goal, episode, step	Per-step state snapshots for replay and debugging

Use aiosqlite for all DB operations to prevent I/O from blocking the RL training loop. Batch write operations: queue events in memory and flush every 500ms via asyncio.

4. Observation & Action Space Design

4.1 Observation Space

Correction from original draft: 84x84 grayscale was suggested as the observation frame size. This is the Atari benchmark standard from 2013 — completely inappropriate for Resident Evil.

RE's gameplay depends heavily on color cues: the red of healing herb pickups, the green of safe doors vs red of locked doors, the blue flash of a parriable enemy attack, blood indicators of damage direction. Downscaling to 84x84 destroys spatial detail needed to distinguish a door from a wall. Use 224x224 RGB.

Component	Shape / Type	Values	Notes
Frame stack	(4, 224, 224, 3) uint8	0–255 RGB	4 most recent frames; temporal motion information at proper resolution
Health	float32 scalar	0.0–1.0	Normalized from EasyOCR HUD read
Ammo primary	float32 scalar	0.0–1.0	Normalized by max capacity for current weapon
Ammo secondary	float32 scalar	0.0–1.0	Normalized
Goal encoding	(16,) float32	0.0–1.0	Learned embedding of current LLM high-level goal category
Room ID encoding	(64,) float32	0 or 1	One-hot of current room; unknown rooms map to all-zeros vector
Visible enemy count	int (0–8)	0–8	YOLO detection count in current frame
Nearest enemy dist	float32 scalar	0.0–1.0	Normalized bounding box centroid distance

4.2 Action Space — MultiDiscrete

The action space is MultiDiscrete — an independent discrete choice for each control dimension. This is significantly more sample-efficient than a flat Discrete space across all 2,160 possible combinations, and it mirrors how RE games are actually controlled on a keyboard.

Dimension	Options	Values	Notes
Movement	9	STOP, FWD, BACK, LEFT, RIGHT, FWD_L, FWD_R, BACK_L, BACK_R	8-directional movement + stop

Dimension	Options	Values	Notes
Camera / Look	5	NONE, LOOK_L, LOOK_R, LOOK_UP, LOOK_DOWN	Discrete steps; pydirectinput.moveRel(± 15 , 0) or (0, ± 15)
Interact	2	NONE, INTERACT	E key — doors, items, levers, switches
Combat	4	NONE, AIM, SHOOT, AIM_AND_SHOOT	RMB = aim, LMB = shoot; AIM_AND_SHOOT for quick shots
Evasion	3	NONE, DODGE, SPRINT	Space = dodge roll, Shift = sprint
Inventory	2	NONE, OPEN_INVENTORY	Tab key — opens/closes inventory screen

Total action combinations: $9 \times 5 \times 2 \times 4 \times 3 \times 2 = 2,160$. SB3's RecurrentPPO handles MultiDiscrete natively with a separate policy head per dimension.

Note on continuous camera control: RE games use analog mouse input. The agent simulates this by calling pydirectinput.moveRel() with small fixed increments. If precise camera aiming proves critical for combat quality in later training phases, the camera dimension can be upgraded to a continuous action using SAC instead of PPO — but start with discrete.

5. Reward System Design

Reward shaping is the most game-specific and most important part of the RL setup. RE games have extremely sparse natural rewards (completing areas, solving puzzles). Without dense intermediate rewards, the agent will spend millions of steps doing nothing. Every reward value below is chosen to maintain a roughly -200 to +500 range, keeping gradient magnitudes stable for PPO.

5.1 Reward Table

Event	Reward	Trigger Condition	Rationale
New room discovered	+50	Template match detects a room not in SQLite rooms table	Primary exploration incentive — this is the core learning signal early on
Key item collected	+75	Item classified as objective-critical (key, emblem, cog, medallion)	High reward to prioritize story progression items
Common item picked up	+15	Inventory count increases, YOLO pickup animation detected	Reward resource gathering; crucial for survival
Standard enemy killed	+30	Enemy YOLO bounding box disappears after combat engagement	Encourage area clearing for safe exploration
Boss phase transition	+150	Boss HP bar template match detects phase decrease	Highest per-event reward for major combat milestones
Puzzle solved	+200	Puzzle UI closes with success state template match	Second-highest reward; puzzles unlock critical story paths
Chapter/area completed	+500	Loading screen transition + area completion template	Max reward; entire session is built around achieving this
Damage taken	-10 per %	EasyOCR detects health bar decrease	Strong incentive to avoid hits; applies per 1% health lost
Death	-250	Game over screen template match	Severe penalty exceeds most session gains — survival is mandatory
Missed shot (no enemy)	-0.5 per shot	SHOOT action fired with no enemy in YOLO detection zone	Discourage ammo waste; conserving ammo is critical in RE
Stuck penalty	-1.0 per step	Same room, no new events for >30 consecutive seconds	Break exploration stagnation loops
Survival step bonus	+0.1 per step	Agent is alive and not stuck	Small baseline keeps the agent active and exploring

5.2 Curriculum Reward Schedule

Apply a curriculum multiplier to guide the agent through learning phases in order:

- Steps 0–50K (Exploration phase): Exploration reward x2.0, Combat reward x0.3. Teach the agent to move around before fighting.
- Steps 50K–200K (Combat introduction): Exploration reward x1.5, Combat reward x1.0. Begin training combat alongside exploration.
- Steps 200K+ (Full game): All rewards at x1.0. Agent should have enough exploration experience to handle full reward signal.

Reward monitoring: Display the decomposed reward breakdown (exploration vs. combat vs. survival vs. penalty) as separate chart lines in the dashboard. If exploration reward exceeds 80% of total for more than 100 episodes, the agent is likely stuck in a loop — inspect its behavior in the live feed.

6. Live GUI Dashboard

6.1 Technology: Gradio 5

Gradio 5 is the recommended dashboard framework. It is Python-native (no JavaScript required), has first-class streaming support for real-time image and text updates, runs as a local web server you open in any browser, and is designed specifically for AI applications — including live streaming of model thoughts and metrics.

- Library: gradio $\geq$ 5.0
- `gr.Image(streaming=True)` for live annotated game feed
- `gr.Textbox(streaming=True)` for real-time LLM thought output as tokens arrive
- `gr.Plot` for live Plotly reward/training charts
- `gr.Accordion` for all expandable/collapsible panels — the core of the 'expand to see everything' requirement
- `gr.Timer` for polling the shared message queue every 100ms and updating all components

6.2 Dashboard Panel Layout

ALWAYS-VISIBLE TOP ROW (3 columns):

[LIVE GAME FEED]	[AGENT STATUS]	[EPISODE METRICS]
Annotated screenshot with YOLO boxes & OCR health/ammo overlays drawn on each frame	Current LLM goal Threat level (0-10) Confidence score Health bar / ammo HUD LLM call countdown	Episode reward total Rooms visited count Deaths this episode Items found / enemies killed this run

EXPANDABLE PANELS (gr.Accordion — click header to expand/collapse):

▶ LLM THOUGHTS STREAM	Streaming token output of LLM reasoning as it types
▶ RL AGENT DECISIONS	Action probabilities, Q-values, entropy, last 50 actions
▶ REWARD & TRAINING CHARTS	Live Plotly: reward curve, loss, episode length over time
▶ MEMORY MAP	Graph of discovered rooms, items, locked doors, bosses
▶ FULL EVENT LOG	Timestamped chronological log of all events and LLM calls

6.3 Real-Time Streaming Architecture

The dashboard communicates with the running agent via an `asyncio.Queue` message bus. The agent writes JSON messages to the queue; Gradio reads and dispatches them every 100ms via `gr.Timer`. This prevents the dashboard from ever blocking the training loop.

- Agent writes: annotated frames (JPEG 80%), LLM response JSON, reward step data, action distribution tensor, event strings

- Dashboard reads: `gr.Timer` polls queue, unpacks message type, routes to the correct `gr.component` update
- Screenshot streaming: encode frames with `cv2.imencode('.jpg', frame, [cv2.IMWRITE_JPEG_QUALITY, 80])` before queuing — reduces bandwidth from 6 MB/frame (raw) to ~150 KB
- LLM streaming: use OpenAI and Anthropic streaming APIs; yield tokens to `gr.Textbox` token-by-token as they arrive from the API — the user sees the LLM reasoning in real time
- Chart updates: Plotly figures are rebuilt from SQLite queries every 5 seconds, not every step — rebuilding on every step would cause UI jitter

7. Implementation Phases

The project is divided into 7 phases. Each phase is independently testable — you should have a working, demonstrable component at the end of each phase before moving forward. Do NOT skip ahead to the next phase before validating the current one.

Phase 1: Environment Setup & Capture Foundation

Estimated time: 1–2 days.

Task	Who	Step-by-Step Notes & Pitfall Warnings
Install Python 3.11 + CUDA Toolkit	YOU	Use Python 3.11 specifically — SB3 2.3+ and Ultralytics both test on 3.11. Avoid 3.12 (some RL dependencies lag behind). CUDA Toolkit must match your GPU driver version — check nvidia-smi output first.
Install PyTorch with CUDA support	YOU	PITFALL: Do NOT run plain 'pip install torch' — this installs the CPU-only build. Always run: pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121 (replace cu121 with your CUDA version)
Install all project libraries	AI (Claude/Copilot)	Ask AI to generate requirements.txt with pinned versions. Key packages: mss, pydirectinput, easyocr, ultralytics, stable-baselines3, sb3-contrib, gradio, aiosqlite, langgraph, openai, anthropic, plotly, python-dotenv, opencv-python
Configure RE game settings	YOU	In-game: Options > Display > Window Mode > Borderless Windowed. Set resolution to your monitor's native. Disable V-Sync for maximum frame rate during training.
Write and test screen capture loop	AI (Claude/Copilot)	Ask AI: 'Write a Python mss screen capture loop that targets a specific window region by pixel coordinates, outputs BGR NumPy frames, and prints achieved FPS.' Run it while the game is open and verify 60 fps.
Write and test pydirectinput control layer	AI (Claude/Copilot)	Test EACH key action individually in-game before moving on. Common pitfall: pydirectinput.press() is instant — some RE actions need pydirectinput.keyDown() / keyUp() held for a duration to register.
Validate full capture→display pipeline	YOU	Run the capture loop, display frames in an OpenCV window (cv2.imshow), confirm 60 fps with no visible lag or color channel errors (green grass should look green, not blue).

Phase 2: Perception Pipeline

Estimated time: 2–3 days.

Task	Who	Step-by-Step Notes & Pitfall Warnings
Run YOLOv11n on live game frames	AI + YOU	AI writes the inference loop. YOU verify that detections look correct while watching the overlay. PITFALL: YOLO outputs in (x, y, w, h) format — convert to pixel coordinates using frame dimensions before drawing bounding boxes.
Set up EasyOCR for HUD region reading	AI (Claude/Copilot)	Identify HUD pixel coordinates for your specific game and resolution (hardcode in config.yaml). Ask AI to write a cropped OCR function that reads health and ammo and returns structured dict. Test by comparing OCR output vs what you see in-game.
Build OpenCV template library for room/state detection	YOU + AI	YOU take screenshots of: each unique room background, the main menu, inventory open, map open, death screen, loading screen, puzzle UIs. AI builds the template matching pipeline using cv2.matchTemplate with TM_CCOEFF_NORMED.
Build event detector with debouncing	AI (Claude/Copilot)	Events must debounce — a 'new room' event should fire ONCE per room visit, not every frame the room template matches. Ask AI to implement hysteresis: match must hold for 10 consecutive frames before triggering an event.
Integration test: full perception pipeline on live gameplay	YOU	Watch the annotated overlay in real time during ~15 minutes of manual gameplay. Verify: enemy boxes appear on enemies, health value matches what you see, room changes are detected. Fix any mismatches before Phase 3.

Phase 3: RL Agent Foundation

Estimated time: 3–5 days.

Task	Who	Step-by-Step Notes & Pitfall Warnings
Build custom Gym environment wrapping the game	AI (Claude/Copilot)	This is the most critical code in the project — the gym.Env subclass with step(), reset(), and render(). Carefully review every line AI generates. PITFALL: reset() must actually reload the game save state (via F9 quick load or game reset menu), not just return a new observation from the current state.
Implement observation space assembly	AI + YOU	AI writes the code; YOU tune normalization values using real game observations. Run 100 manual steps and print observation stats — verify health is truly 0–1 normalized, ammo is correctly scaled.

Task	Who	Step-by-Step Notes & Pitfall Warnings
Implement reward function (exploration-only curriculum first)	AI + YOU	Start with ONLY room discovery reward and death penalty. Add combat and item rewards in Phase 7. This intentionally forces the agent to explore before fighting — prevents getting stuck in combat loops.
Write non-interactive state detector (cutscenes, menus, death)	AI (Claude/Copilot)	CRITICAL. The RL agent MUST return a zeroed observation + zero reward + not advance step counter during cutscenes and loading screens. Test this explicitly: trigger a death, verify the environment correctly enters paused mode.
First training run: 10,000 steps	YOU	Do NOT expect good behavior. Success criteria: no Python exceptions, reward is trending upward (not just flat at -250 deaths), agent is taking actions (not freeze-looping). Check the SB3 tensorboard logs.
Tune PPO hyperparameters with AI guidance	AI (Claude) + YOU	Share your first training log (reward curve, loss curves) with Claude and ask for hyperparameter recommendations. Key knobs: n_steps (2048 default, try 4096), learning_rate (3e-4 default), ent_coef (0.01, increase if agent stops exploring).

Phase 4: LLM Brain Integration

Estimated time: 2–3 days.

Task	Who	Step-by-Step Notes & Pitfall Warnings
Set up API credentials	YOU	Create a .env file in the project root with OPENAI_API_KEY=..., ANTHROPIC_API_KEY=..., XAI_API_KEY=... Use python-dotenv to load them. NEVER hardcode API keys. Add .env to .gitignore immediately after creating it.
Build LangGraph decision workflow	AI (Claude/Copilot)	Share the workflow graph description from Section 3.4 with AI and ask it to implement the full LangGraph pipeline. Test the pipeline by sending 10 real in-game screenshots manually and inspecting JSON output quality.
Build and test all prompt templates	AI (Claude) + YOU	Start with the new_room and stuck templates only. Run live tests and compare LLM decisions to what YOU would do in the same situation. Rewrite prompts where LLM makes consistently poor decisions.
Wire async LLM calls to event detector	AI (Claude/Copilot)	LLM calls run in a concurrent.futures.ThreadPoolExecutor. The event detector fires a trigger; the thread pool picks it up asynchronously. The RL loop NEVER awaits an LLM response.
Goal injection into RL observation	AI (Claude/Copilot)	Parse the rl_sub_goal string from the LLM JSON response, map it to a predefined goal category index

Task	Who	Step-by-Step Notes & Pitfall Warnings
		(explore, combat, flee, interact, inventory), encode as one-hot, and write to a shared goal_vector that the env.step() observation assembler reads.

Phase 5: Memory System

Estimated time: 1–2 days.

Task	Who	Step-by-Step Notes & Pitfall Warnings
Create SQLite database and schema	AI (Claude/Copilot)	Ask AI to generate the CREATE TABLE statements from Section 3.5's schema and an async helper module using aiosqlite. Verify all tables are created correctly by opening the .db file in DB Browser for SQLite.
Wire perception events to database writes	AI (Claude/Copilot)	Every event detector trigger writes a row to the events table asynchronously. PITFALL: Do NOT await individual DB writes in the hot path — queue them and batch flush every 500ms.
Build room graph serializer for LLM context	AI (Claude/Copilot)	Query the rooms table, build an adjacency list of connected rooms, serialize to compact JSON string, and inject into the CONTEXT BLOCK of each LLM prompt. This gives the LLM a map of what the agent has discovered.
Validate data quality with manual play test	YOU	Play the game manually for 30 minutes while the system runs. Then open DB Browser and inspect every table. Ask: are events logged? Are rooms tracked? Are items recorded? Fix any gaps before proceeding.

Phase 6: GUI Dashboard

Estimated time: 2–3 days.

Task	Who	Step-by-Step Notes & Pitfall Warnings
Build asyncio.Queue message bus	AI (Claude/Copilot)	Single shared queue object. Message format: {'type': str, 'payload': dict}. Message types: 'frame', 'llm_token', 'llm_complete', 'reward', 'action_probs', 'event'. AI writes the queue; you verify messages flow correctly with a test producer.
Build Gradio 5 layout	AI (Claude/Copilot)	Share the layout spec from Section 6.2 with AI and ask it to build the full Gradio app structure. Verify the layout in a browser before wiring any live data to it.

Task	Who	Step-by-Step Notes & Pitfall Warnings
Implement live game feed with YOLO overlay	AI (Claude/Copilot)	Use gr.Image component with a gr.Timer that polls 'frame' messages from the queue. PITFALL: Gradio's gr.Image expects a NumPy array or file path — make sure JPEG decode is happening before passing to the component.
Implement LLM thought stream	AI (Claude/Copilot)	Use OpenAI/Anthropic streaming APIs and yield partial text to a gr.Textbox via generator — this creates the 'typing' effect. Prepend each new call with a timestamp divider for readability.
Implement Plotly reward charts	AI (Claude/Copilot)	Query SQLite events table for step rewards every 5 seconds. Build a rolling 1,000-episode window Plotly figure. Update via gr.Timer. Include separate traces for: total reward, exploration reward, combat reward, death count.
Full integration test with live agent	YOU	Run the complete system with the game open. Verify all panels update in real time. Test every gr.Accordion expand/collapse. Confirm no memory leaks over 30+ minutes of continuous operation.

Phase 7: Training, Optimization & Continuous Iteration

Estimated time: Ongoing. This phase never truly ends — it is the long-term iteration loop.

Task	Who	Step-by-Step Notes & Pitfall Warnings
Initial 100K-step training run	YOU + AI	Run overnight. Review reward curve and event log in dashboard. Share training log with AI for diagnosis if reward stagnates or diverges.
Expand reward function to include combat & items	AI + YOU	Add all rewards from the reward table now that the agent has basic exploration. Monitor for reward hacking (agent exploiting a single reward source).
Fine-tune YOLOv11 on RE-specific frames	HIRE or AI+YOU	Collect 300–500 gameplay frames with enemies, items, and doors visible. Annotate using LabelImg. Ask AI to write the fine-tuning script. Run the fine-tuning job overnight on your GPU. Expect ~2–3 hours for 300 images.
LLM prompt optimization	YOU + AI	Review the LLM Thoughts panel in the dashboard. Identify the 5 most common poor LLM decisions from the event log. Rewrite the prompt templates for those specific failure cases. Test each change with 10 representative screenshots.
Hyperparameter optimization with Optuna	AI (Claude/Copilot)	Ask AI to set up an Optuna study that tunes learning_rate, n_steps, ent_coef, gamma. Run 20 trials. Apply the best hyperparameters and run a fresh training from checkpoint.

Task	Who	Step-by-Step Notes & Pitfall Warnings
Multi-game generalization	YOU + AI	After training on one RE game, test the perception pipeline on a second game. Expect to need: new room templates, adjusted HUD coordinates in config.yaml, and potentially a new YOLO fine-tune. The RL policy should transfer partially.

8. Key Implementation Challenges & Solutions

8.1 Screen Capture Latency

Challenge: The agent's observation must be current enough to react to fast game events (incoming attacks, projectiles). Each millisecond of additional latency reduces the quality of combat learning.

- Solution: Dedicated capture thread running at monitor refresh rate. A shared deque(maxlen=1) holds the latest frame — inference always reads the newest frame without waiting.
- Target: <30ms total pipeline latency (capture → perception → action). Benchmark each stage separately during early development.
- Solution: Separate PyTorch inference (YOLO) onto a dedicated CUDA stream from the RL training compute graph, preventing GPU memory bandwidth contention.

8.2 Reward Sparsity and Exploration Stagnation

Challenge: RE games have very long paths between natural reward events. A naive RL agent will wander in circles without discovering that opening a door leads to a reward.

- Solution: Curiosity-driven exploration via Random Network Distillation (RND). RND gives intrinsic rewards for visiting novel states, independent of game rewards. Install via rnd-sb3 for SB3 compatibility.
- Solution: Use the LLM brain's goal injection to push the agent toward unexplored regions when it is stuck. This extrinsic guidance amplifies intrinsic curiosity at critical bottleneck moments (locked doors, unexplored corridors).

8.3 Game State Resets for RL Training

Challenge: RL requires thousands of death-and-restart cycles. Manually navigating through menus to reload a save would take minutes per reset, making training impractical.

- Solution: Use RE's quick save (F5) and quick load (F9) hotkeys, automatable via pydirectinput. The env.reset() method: presses F9, waits for loading screen to clear (template match confirms), then returns the first observation. Full reset takes ~8–15 seconds.
- Solution: Maintain multiple save files for different training stages. Train early RL in a simple starting room (one save), then advance to multi-room scenarios as the agent improves.

8.4 Observations During Non-Interactive States

Challenge: Loading screens, cutscenes, and death animations produce meaningless frames that corrupt RL training — the agent learns to associate loading screen pixels with random rewards.

- Solution: Non-interactive state templates detect these states via `cv2.matchTemplate`. When matched: return `np.zeros_like(last_valid_obs)` as the observation, return 0.0 reward, set `done=False`, and do NOT advance the step counter. The RL trainer effectively skips these frames entirely.

8.5 LLM Hallucination and Poor Decision Quality

Challenge: LLMs can hallucinate game mechanics, suggest impossible actions, or misread in-game state from ambiguous screenshots.

- Solution: Validate all LLM outputs against known game state before acting on them. If LLM says 'use herb' but no herbs exist in the inventory (from SQLite items table), discard that instruction and retain the previous goal.
- Solution: Confidence gating — if confidence < 0.4 in the LLM JSON response, trigger a second LLM call with additional context before committing to a goal change. Log both calls to the event database for debugging.
- Solution: Include the last 3 LLM decisions + outcomes (did following that goal earn positive reward?) in the context block. This gives the LLM a feedback loop on its own quality.

8.6 GPU Memory Pressure

Challenge: Running the game, YOLOv11 inference, RL training, and video streaming simultaneously may push GPU memory limits on consumer hardware.

- Solution: Always use YOLOv11n (nano variant — ~6MB VRAM). The larger s/m/l models offer marginal accuracy gains in exchange for dramatically higher VRAM use at inference time.
- Solution: LLM calls (GPT-4o, Claude) are cloud API calls — they use Anthropic's and OpenAI's infrastructure, not your local GPU. Zero local VRAM impact.
- Solution: Enable mixed precision training in SB3 via `policy_kwargs={'optimizer_class': torch.optim.AdamW}` and wrapping the training loop with `torch.cuda.amp.autocast()`. Halves VRAM footprint of the policy network.
- Minimum hardware: 8 GB VRAM (RTX 3070 minimum). 12 GB VRAM (RTX 3080 / RTX 4070) recommended for comfortable headroom during simultaneous game + training.

9. Best Practices & Common Pitfalls

9.1 Project Structure Best Practices

- Use a single config.yaml file for all magic numbers: window coordinates, reward values, HUD crop regions, LLM cooldown thresholds, template match thresholds. Never scatter hardcoded values through code.
- Checkpoint RL model every 10,000 steps using SB3's CheckpointCallback. Losing 200K+ training steps to a crash or power failure is catastrophic and entirely avoidable.
- Store all API keys in a .env file loaded via python-dotenv. Add .env, checkpoints/, logs/, *.db, and __pycache__ to .gitignore before the first commit.
- Use git version control from Day 1. Commit a working build at the end of each phase before starting the next. Branch for experimental reward changes so you can revert if training degrades.
- All pixel coordinates must use relative values (percentage of capture region width/height) in config.yaml — never absolute pixel numbers. This makes the system work at any resolution without code changes.

9.2 RL-Specific Pitfalls to Avoid

PITFALL: Reward hacking — the agent discovers an unintended way to maximize reward. Classic example: agent stands next to a healing item and picks it up before it fully consummates, triggering the pickup reward repeatedly without actually healing.

Solution: Add a per-item-per-session pickup flag in SQLite. Items already collected in this episode cannot trigger the pickup reward again. Reset flags at episode start.

PITFALL: Observation normalization breaks after a game update that moves a HUD element. The agent suddenly receives nonsense health values and combat training collapses.

Solution: Run a nightly sanity check script that compares observed health/ammo values from EasyOCR against known valid ranges. Alert via dashboard if values are outside bounds for >100 consecutive steps.

PITFALL: Catastrophic forgetting — as the agent learns a new area's mechanics, it 'forgets' older area behaviors because those experiences are underrepresented in recent replay buffers.

Solution: Use a balanced experience replay strategy with SB3 — sample experiences uniformly from ALL episodes in the replay buffer, not just recent ones. This is controlled via the buffer_size and batch_size parameters.

9.3 LLM Integration Pitfalls

PITFALL: LLM API rate limits. GPT-4o allows ~500 requests/minute on the paid tier. With a 3-second cooldown, you will use at most 20 requests/minute — well within limits. However, if you remove the cooldown during debugging, you can hit limits immediately.

Solution: Track cumulative LLM calls per hour in the dashboard. Set a soft warning at 300 calls/hour and a hard cap at 450 calls/hour (leave headroom for burst events).

PITFALL: Context window overflow on long sessions. Event histories grow indefinitely if not managed. A 2-hour gaming session can accumulate 1,000+ events, exceeding the LLM context window.

Solution: Summarize events older than 50 entries using a cheap GPT-4o-mini call: 'Summarize these 50 game events in 3 sentences.' Replace the detailed list with the summary in the LLM context. Keep the last 10 events in full detail.

10. Task Division Guide

Based on the project's complexity and the tools available, here is a clear division of who should handle each type of work:

Tasks You Should Do Yourself

These require game knowledge, subjective judgment about quality, or direct interaction with physical hardware:

- Configure the game to borderless windowed mode and validate the capture pipeline visually
- Take and label initial YOLO training screenshots (you know what enemies, items, and doors look like)
- Review LLM decisions in the Thoughts panel and identify which ones are wrong or suboptimal
- Tune reward values based on observed agent behavior — increase/decrease specific rewards by watching 30-minute training clips
- Set up your Python environment, install CUDA toolkit, create .env with your API keys
- Monitor ongoing training runs, decide when reward stagnation needs intervention vs. natural plateau
- Validate the perception pipeline accuracy by watching the overlay during live gameplay

Use AI Coding Tools (Claude / Copilot / GPT-4) For

These are well-defined coding tasks where AI is extremely effective. For all of these, describe what you want clearly, review what is generated, and test it:

- Writing all Python code for every component (capture loop, perception pipeline, Gym environment, LangGraph workflow, Gradio dashboard, SQLite helpers)
- Debugging Python errors — paste the full stack trace and ask for a fix and explanation
- Writing all prompt templates and iterating on them based on your feedback from real LLM calls
- Generating the YOLO fine-tuning training script and Optuna hyperparameter optimization study
- Analyzing training logs and tensorboard output to suggest hyperparameter adjustments
- Writing all unit tests and integration tests for each component
- Refactoring code as the project grows — keep modules clean as the codebase scales

Consider Hiring a Specialist For

These tasks exceed typical AI coding assistance or require specialized expertise that would take significant personal time to develop:

- Large-scale YOLO dataset creation (>500 labeled images): A computer vision data labeling service or a part-time data annotator can build a professional-grade RE-specific dataset far faster than manual labeling
- DirectX game memory reading (if you want to read health/ammo directly from game RAM instead of via OCR): This requires low-level reverse engineering expertise
- Cloud GPU training infrastructure (AWS/Azure/GCP spot instances for 24/7 training at scale): A DevOps engineer or MLOps specialist can set this up in a day
- Distribution as a polished desktop application (replacing Gradio with a native Windows app): A frontend developer with Python/Electron experience if you want to share this as a product

11. Performance Requirements & Hardware

Component	Minimum	Recommended	Notes
GPU VRAM	8 GB	12–16 GB	RTX 3070 minimum; RTX 3080 / RTX 4070 Super recommended
System RAM	16 GB	32 GB	Frame buffers, SQLite cache, Python runtime overhead
CPU	6-core / 3.5 GHz	8-core / 4.5 GHz+	Perception pipeline + mss capture are CPU-intensive
Storage	100 GB SSD	500 GB NVMe SSD	RL checkpoints + screenshot logs fill quickly over long runs
Internet	25 Mbps	100 Mbps+	LLM API calls; lower bandwidth increases per-call latency spikes
Capture FPS	30 fps	60 fps	Higher = better temporal resolution = better combat reflexes
Perception latency	<50 ms	<20 ms	YOLOv11n on RTX 3080 achieves ~4–5 ms
Full pipeline latency	<100 ms	<33 ms	Capture + perception + action execution end-to-end
LLM call frequency	Max 20/min	Max 15/min	With 3-sec cooldown and event throttling

12. Testing Strategy

12.1 Component Tests (Before Any Integration)

- Capture: Run 60 seconds of capture, verify sustained ≥ 30 fps, verify correct frame dimensions and BGR color channels (print a pixel value from a known colored game element to confirm)
- Perception: Run YOLO on 100 manually labeled test frames, verify average precision ≥ 0.5 before integrating with RL
- Input: Write a test script that executes every action key and manually verify each one registers in-game
- LLM: Send 10 representative RE screenshots to each provider; manually rate response quality out of 10; require average ≥ 7 before integration
- Database: Simulate 10,000 async event writes, verify all rows present, verify concurrent read/write causes no corruption

12.2 Integration Tests (Before Starting Training)

- Run a 1,000-step episode with all components active — verify no crashes, verify dashboard updates in real time, verify LLM calls fire on trigger events
- Manually trigger every LLM trigger condition and verify LLM call fires correctly (walk to a puzzle, take damage to $<25\%$ health, stand still for 31 seconds)
- Kill the game process mid-training — verify the system recovers cleanly: exception is caught, episode ends, environment resets, training resumes from last checkpoint
- Verify database persistence: stop and restart the agent, confirm previous room knowledge and event history are loaded from SQLite at startup

12.3 Training Milestone Validation

1. 10K steps: No crashes. Episode rewards should be non-trivially positive (trending above -250 baseline). Agent takes varied actions.
2. 50K steps: Agent explores at least 3–5 distinct rooms per episode. Exploration reward is the primary reward component.
3. 200K steps: Death rate declining trend. Agent shows basic threat avoidance (moving away from enemies consistently).
4. 500K steps: Agent completes at least the first chapter area consistently. LLM goals are being followed $>40\%$ of the time (measure via event log).
5. 1M+ steps: Agent demonstrates learned strategies visible in the live feed: cornering enemies before shooting, prioritizing healing when health is low, following LLM puzzle guidance to completion.

END OF SPECIFICATION — v1.0